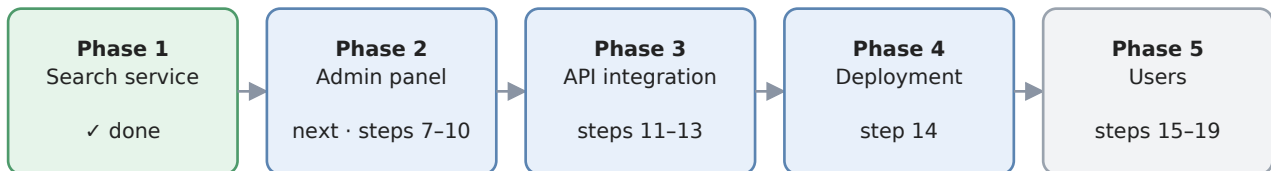


Next plan and your decisions

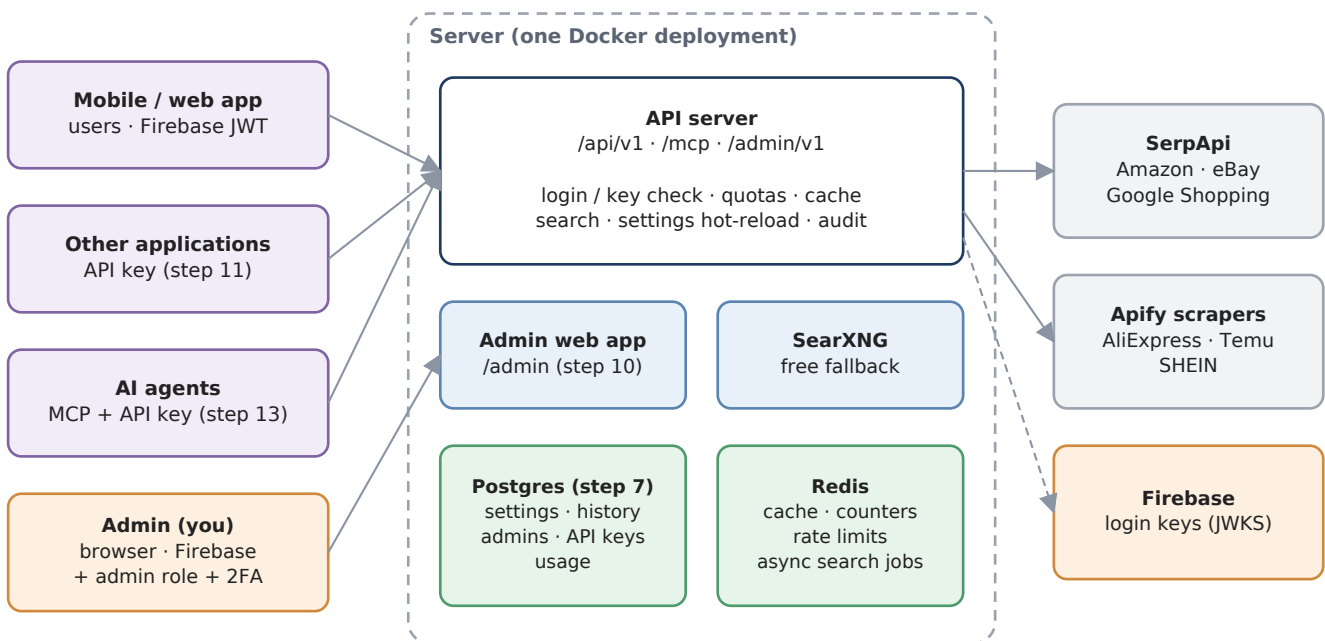
owis_find_deal_engine — admin panel, API integration for other applications and AI agents, deployment. What will be built, in which order, and what is needed from you.



Owner's new order (7 Oct 2026): admin panel first, then other applications / AI agents, then deployment; user features after.

Needed from you before step 7 starts: answers to decisions D1-D4 (page 4). D5-D8 are needed later, at the step named next to them. Each decision has a recommendation — answering "recommended" is enough.

Where we are going



After phases 2-4: three kinds of callers, one server, settings in a database.

Phase 2 — Admin panel

Goal: control every setting of the service from a browser, safely, without editing files or restarting the server.

Step	What is built	Result for you
7. Database + settings store	Postgres joins the stack. Tables: settings, settings history (who, when, old → new), admins. Runtime settings move from .env to the database; the server applies a change by rebuilding its search sources and switching over without a restart.	change a setting → active in seconds; every change can be traced and undone
8. Admin API (/admin/v1)	Login with Firebase + admin role, checked on every call; audit log; rate limit. Settings: countries and shops, source per shop (scraper, input, currency, result limit, timeout, max cost), SerpApi engines, cache times, plans, quotas, rate limits, CORS. Secrets write-only and encrypted. Tools: test search (each source's answer, time, cost), clear cache, source health.	everything in today's .env controllable, safely
9. Usage and cost dashboard	Live searches, cache hit rate, fallbacks and errors per source, estimated cost per day (Apify runs × price, SerpApi searches), top searches; alert at a monthly budget.	see what the service costs before the bill arrives
10. Admin web app	Browser app: settings forms with validation, dashboard, audit log, API clients. Served by the same server under /admin.	one address, nothing extra to host

Phase 3 — API integration (other applications and AI agents)

Step	What is built	Result for you
11. API clients and keys	Created in the admin panel: name, scopes (search, countries), daily quota, rate limit, allowed countries, optional IP list, expiry. Key shown once (ofd_live_...); only a hash is stored. Revoke and rotate. Usage counted per client.	give a partner or agent access in a minute, take it back at once
12. Async search + webhooks	POST /api/v1/searches → 202 + id; GET /api/v1/searches/{id}; optional signed webhook when done (live searches take 25-50 s, too long for many clients).	integrations do not time out
13. AI agents	MCP server endpoint (/mcp) with tools search_products and list_countries, same API keys; OpenAPI 3.1 spec + docs page.	Claude and other agents can use the service directly

Phase 4 — Deployment of everything

Step	What is done
14. Deployment	Hosting, domain, HTTPS (Caddy or Cloudflare), private networks for databases, secrets in the host's secret store, Postgres backups, staging + production, automatic deploy from main, monitoring and alerts (searches, cache, provider errors, costs).

Phase 5 — Users (after deployment)

Step	What is built
15	Users table, profile (/me), email sign-in links
16	Plans in the database + payment webhook
17	Saved cart + price tracker (alerts when a saved item gets cheaper)
18	Clicks + purchase reports
19	Notifications: inbox, push (FCM), background jobs

Later (when possible)

- eBay Browse API (free) and AliExpress affiliate API, if the developer keys are approved.
- Another SHEIN scraper: cheaper, or supporting the Saudi / Arab sites (sa, ar, ae).
- App work: loading bar, "approx." price label, AliExpress welcome-deal note.

Your decisions

Recommended options are marked ★. D1-D4 are needed **before step 7**.

#	Decision	When	Options
D1	Admin web app technology	before step 7	★ A: small React web app built into the server image (one deploy, modern forms) B: simple server-rendered pages (no JavaScript build, plainer look)
D2	Who can be admin, and how they log in	before step 7	★ A: your Firebase account + admin list in the database + two-factor login required B: Firebase account + admin list, without two-factor
D3	Secrets (SerpApi key, Apify token)	before step 7	★ A: changeable in the panel — encrypted, write-only, never shown again B: only in .env on the server (panel shows "set / not set")
D4	Who will use the API (phase 3)	before step 7 (shapes the data model)	★ A: your own apps and agents first; keys created by you in the panel B: also outside partners / developers → later self-service sign-up and billing
D5	Monthly budget alert	step 9	an amount for Apify + SerpApi per month (e.g. \$5 Apify free credit + 250 SerpApi searches)
D6	Hosting	step 14	★ A: one small VPS (Hetzner / DigitalOcean, ~\$5-8 / month) with Docker B: Fly.io / Render / Railway + managed Postgres (Neon / Supabase) C: Oracle Cloud Always Free (free, strict sign-up)
D7	Domain name	step 14	the domain for the API and admin panel, e.g. api.yourdomain.com / admin.yourdomain.com
D8	Paid plans for Apify / SerpApi	before real users	free credit covers testing only (≈ 20 live searches / month for Apify)

Answer sheet (copy into the next session)

D1 admin web app: A / B
 D2 admin login: A / B admin email(s): ...
 D3 secrets: A / B
 D4 API users: A / B
 D5 budget alert: Apify \$... / month, SerpApi ... searches / month (can wait)
 D6 hosting: A / B / C (can wait)
 D7 domain: ... (can wait)
 D8 paid plans: ... (can wait)

Next session: open the repository and say "Continue the Owis_Find_Deal_Engine project from Owis_Find_Deal_Engine/STATUS.md on branch claude/cloud-vs-local-7xi988. Start phase 2, step 7. My answers: D1 ..., D2 ..., D3 ..., D4 ..."